Practical 5: Resources, Intents, and Dynamic UI in Android

1. Learning Objectives

By completing this practical, you should be able to:

- Create and organise Android resources, including drawables, launcher icons, strings, colours, and string arrays.
- Design Android user interfaces using XML layouts and common UI components.
- Create multiple Activities in an Android application.
- Navigate between Activities using explicit Intents and pass data between them.
- Create and use a regular Java class to validate and process user input.
- Access and modify UI components from Java code using `findViewById()`.
- Handle keyboard input and button click events.
- Use an implicit Intent to open an official municipality website in a suitable application.
- Apply conditional logic to customise an application's content and appearance according to user input.

2. Scenario

Develop an Android application called `Chinese Municipalities`. The application allows users to enter the name of one of China's four municipalities: `Beijing`, `Shanghai`, `Tianjin`, or `Chongqing`. The application validates the input, displays information about the selected municipality, and customises the screen with the appropriate image and colour scheme. Users must also be able to open the official website of the selected municipality.

3. Create the Android Project

1. Open Android Studio and create a new project using a suitable Empty Views Activity template.
2. Set the project name to `PracticalFive`.
3. Select Java as the programming language and use XML layouts for the UI.
4. Allow Android Studio to generate the initial project files.

4. Required Application Resources

4.1 Images

Place the following images in res/drawable/. Use the supplied images where provided. If an image is not supplied, obtain a suitable image and ensure you have permission to use it.

Image file	Purpose
`chinese_municipalities.jpg`	Image displayed on the main screen
`beijing.jpg`	Image displayed for Beijing
`shanghai.jpg`	Image displayed for Shanghai
`tianjin.jpg`	Image displayed for Tianjin
`chongqing.jpg`	Image displayed for Chongqing

Use lowercase filenames with underscores. Android resource filenames must not contain spaces or uppercase letters.

4.2 Application Name

Open `res/values/strings.xml` and set the `app_name` resource to the following:

```
<string name="app_name">Chinese Municipalities</string>
```

4.3 Launcher Icon

1. Right-click the `res` folder → `New → Image Asset`.
2. Ensure `Icon Type` is set to `"Launcher Icons (Adaptive and Legacy)"`.
3. Name: `ic_launcher` (Do not change this name).
4. Under `Foreground Layer`, change `"Asset Type"` to `Image`, and select `app_logo.png` image. Adjust the resize slider so it fits nicely in the safe zone.
5. Under `Background Layer`, choose a solid color (e.g., Black).
6. Click `Next`, then `Finish`.

4.4 Colour Resources

Create the following colour resources in res/values/colors.xml. If the file already exists, add these resources to it. Choose suitable hexadecimal values for each colour.

Resource name	Required colour
`municipality_beijing`	Red
`municipality_shanghai`	Magenta
`municipality_tianjin`	Charcoal
`municipality_chongqing`	Bright orange
`color_text_light`	A light text colour suitable for dark backgrounds
`color_text_dark`	A dark text colour suitable for light backgrounds
`color_default_background`	A neutral background colour

Use named resources throughout the application. Do not repeat colour hexadecimal values in multiple Java statements.

4.5 Municipality Information Array

Create a values resource file named `arrays.xml`: right-click `res/values`, select `New → Values Resource File`, and enter `arrays` as the filename. Create a string array named `municipalities_array`.

Research each municipality using reliable sources and write a factual summary in your own words. Each summary should be approximately 80–120 words. Store four items in this exact order: `Beijing`, `Shanghai`, `Chongqing`, `Tianjin`.

Example structure:

```
<resources>
    <string-array name="municipalities_array">
        <item>Beijing is the capital … </item>
        <item>Shanghai is one of the … </item>
        <item>Tianjin is a direct-administered … </item>
        <item>Chongqing, formerly romanized …</item>
    </string-array>
</resources>
```

The order matters because your Java implementation must retrieve the correct summary for the selected municipality. `strings`, `colors`, and `arrays` belong under `res/values`; images belong under `res/drawable`; launcher icons belong under `res/mipmap`.

5. Main Screen: MainActivity.java and activity_main.xml

Design the main screen with these three components:

- A TextView displaying "`Municipalities in China`".
- An ImageView displaying `chinese_municipalities.jpg`.
- An EditText with the label "`I want to know about:`".

Main-screen layout reference



Requirements

- Give the EditText a unique ID so it can be accessed from Java.
- Configure the input field as a single-line text input.
- Configure the keyboard so Enter displays an appropriate action, such as `Go` or `Done`.
- When Enter is pressed, retrieve the input and validate it using `TextProcessor`.
- Accept municipality names regardless of case: `beijing`, `BEIJING`, and `Beijing` must all be accepted.
- If the input is empty or unsupported, display an appropriate error and remain on the main screen.
- If valid, pass the normalised name to the second Activity using an explicit Intent.

6. Create the TextProcessor Java Class

Create a regular Java class named `TextProcessor`. It must not extend Activity and must not have an associated XML layout.

1. In the Android Studio Project panel, locate the Java source folder containing your application package.
2. Right-click the package and select `New → Java Class`.
3. Name the class `TextProcessor`.

Implement a method named `processMunicipality(String municipality)`. The method must:

- Accept a municipality name as a String.
- Check whether the input is `null`, empty, or contains only whitespace.
- Return an invalid result for null or empty input (for example, `null`).
- Check whether the input matches one of the four supported municipalities without being case-sensitive.
- Return the correctly capitalised name for valid input.
- Return an invalid result for unsupported names such as `Guangzhou`.

Input	Expected result
`beijing`	Beijing
`SHANGHAI`	Shanghai
`tianjin`	Tianjin
`ChOnGqInG`	Chongqing
`Guangzhou`	Invalid result
`Empty or whitespace only`	Invalid result

`MainActivity` and `Municipality` must handle the invalid result safely. Do not duplicate all validation logic in multiple Activities.

7. Create the Second Activity

Create an Activity named `Municipality` with the associated layout `activity_municipality.xml`.

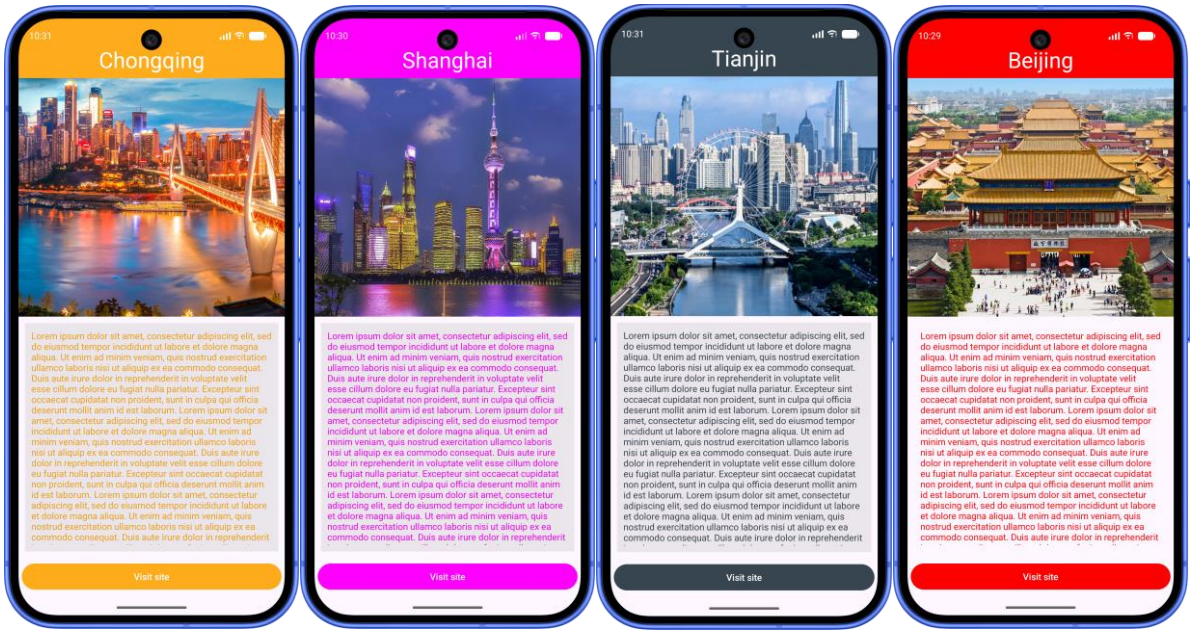
1. Right-click your Java package.
2. Select `New → Activity → Empty Views Activity`.
3. Name the Activity `Municipality`.
4. Confirm that the generated layout is `activity_municipality.xml`.
5. Check that Android Studio has registered the Activity in `AndroidManifest.xml`.

Required UI Components

- A TextView displaying the selected municipality name.
- An ImageView displaying the corresponding municipality image.
- A scrollable information section containing a second TextView.
- A button labelled `Visit site`.

Use a ScrollView around the information TextView. Give it suitable dimensions or constraints so the `Visit site` button remains accessible.

Second-screen layout references



8. Pass and Process the Municipality Name

When the user presses `Enter` on the main screen:

1. Retrieve the text from the EditText.
2. Validate and normalise it using `TextProcessor`.
3. If invalid, display an error and remain on the main screen.
4. If valid, create an explicit Intent targeting `Municipality`.
5. Pass the normalised name as an Intent extra.
6. Start the second Activity.

In `Municipality.java`, retrieve the name passed from `MainActivity.java`, process it using `TextProcessor`, handle missing or invalid data safely, and use the validated name to determine which resources to display. Use the same Intent extra key in both Activities.

9. Customise the Second Activity

The appearance and content of the second Activity must change according to the selected municipality.

Municipality	**Name and `Visit site` button background**	**Information text colour**	**Image resource**
Beijing	Red	Red	`beijing.jpg`
Shanghai	Magenta	Magenta	`shanghai.jpg`
Tianjin	Charcoal	Charcoal	`tianjin.jpg`
Chongqing	Bright orange	Bright orange	`chongqing.jpg`

The name TextView must display the correctly capitalised municipality name. The information TextView must display the corresponding summary from `municipalities_array`.

For example, if the user enters `BEIJING`, the second Activity must display the name `Beijing`, the Beijing summary from `municipalities_array`, `beijing.jpg`, a red background for the name TextView, red information text, and a red `Visit site` button.

Use the named colour resources rather than hard-coding colour values. Ensure text on coloured backgrounds remains readable; use a contrasting text colour, such as white, where necessary.

Your Java implementation should select resources based on the validated municipality name and update the UI programmatically using methods such as `TextView.setText()`, `TextView.setBackgroundColor()`, `TextView.setTextColor()`, `ImageView.setImageResource()`, and an appropriate button background or tint method.

10. `Visit site` Button: Opening the Official Municipality Website

It's a required behavior in this practical to open the official website of the municipality currently displayed. This is intended to demonstrate an implicit Intent.

Municipality	**Suggested official website**
Beijing	`https://english.beijing.gov.cn/`
Shanghai	`https://english.shanghai.gov.cn/`
Tianjin	`https://en.tj.gov.cn/`
Chongqing	`https://english.cq.gov.cn/`

Check that the URLs work before submitting. If a suggested URL is unavailable, research and use a suitable official website for that municipality.

Implementation requirements:

1. In Municipality.java, retrieve the `Visit site` button using `findViewById()`.
2. Register an `OnClickListener` for the button.
3. Determine which municipality is currently displayed.
4. Select the corresponding official website URL.
5. Create an implicit Intent using `Intent.ACTION_VIEW` and the selected URL.
6. Call `startActivity()` to ask Android to open the URL in a suitable application, normally a web browser.
7. Handle the possibility that no suitable application is available to open the URL.

Do not use an explicit Intent targeting another Activity in your application for this requirement. The destination website must depend on the selected municipality.

Expected behaviour: clicking `Visit site` on the Beijing screen opens the Beijing website; clicking `Visit site` on the Shanghai, Tianjin, or Chongqing screen opens that municipality's website.

11. Testing Requirements

Input	Expected result
`Beijing`	Opens Beijing information screen
`beijing`	Opens Beijing information screen
`BEIJING`	Opens Beijing information screen
`Shanghai`	Opens Shanghai information screen
`Tianjin`	Opens Tianjin information screen
`Chongqing`	Opens Chongqing information screen
`Guangzhou`	Displays an invalid municipality error
`Empty input`	Displays an input validation error
`Spaces only`	Displays an input validation error

For each valid municipality, verify that the name, image, history, background colours, information text colour, and `Visit site` button colour are correct. Click `Visit site` for all four municipalities and confirm that each opens the corresponding official website.

Final reminder: Your application must work for all four municipalities, not just Beijing. Use Java for application logic, XML for layouts, and Android resources for reusable text, colours, images, and municipality information.